

Provenance-Aware Capability Gating against Indirect Prompt Injection in Multi-Agent Systems

Chi Kit WONG, Haotian WANG, Guanrun PENG, Boyong HOU
{ckwong627, hwang418, gpeng615, bhou204}@connect.hkust-gz.edu.cn

Abstract—Tool-using agents can confuse data supplied by an attacker with authority granted by the user. We study the smallest auditable version of this problem: ordinary capability gating checks whether a value is on an allow-list, whereas PACT (Provenance-Aware Capability Tracking/Control) also checks whether that value is trusted for its parameter role. We implement a deterministic PACT reference monitor and evaluate four frozen cases. PACT allows a user-selected Alice recipient, denies an externally supplied Bob even when Bob is on the same allow-list, allows external text at a low-risk content role, and allows a user value after an explicitly registered Base64 transformation. Every case records the argument role, value provenance, both decisions, execution and side-effect count, rejection reason, and SHA-256 decision/provenance evidence. The public AgentDojo benchmark is retained only as an external attack-realism baseline; it does not directly evaluate PACT. The result is a small, reproducible mechanism demonstration rather than a claim of semantic information-flow security.

I. INTRODUCTION

Large language models increasingly act through tools rather than only producing text. Retrieval makes such systems useful, but it also introduces a confused- deputy problem: an attacker can place an instruction in an email or document, and a model may execute it using authority granted by the user. This is an *indirect prompt injection* because the adversarial instruction arrives through data selected at run time rather than through the user message [1]. Evaluations of tool-using agents show that neither ordinary helpfulness prompting nor simple refusal instructions reliably separate data from commands [2], [3].

This project asks:

Can a provenance-aware role check prevent an externally supplied allow-listed value from laundering authority into a high-impact parameter, while still allowing benign user values, low-risk external content, and explicitly registered transformations?

We deliberately make a narrow claim. The prototype is a local synthetic office, the capability oracle is assumed correct, and leakage tracking recognizes exact registered

values. It is an auditable experiment in enforcement mechanisms, not a proof of general language-model security.

II. BACKGROUND & MOTIVATION

Prompt-level instruction hierarchy attempts to teach a model that trusted system and user instructions outrank untrusted content [4]. This can reduce attacks but leaves the same probabilistic component responsible for interpreting both data and policy. Capability systems instead provide an unforgeable, least-authority description of the operations a component may perform [5], [6]. Information-flow control tracks where information came from and constrains where it may go [7].

PROVGATE combines these ideas at the tool boundary. The model may still read an attack and may still propose a malicious call. Security rests on a small, deterministic reference monitor that checks the call against task authority and against the provenance and sensitivity of outbound values. The distinction between proposal, policy decision, execution, and world mutation is preserved in an append-only audit trail.

III. TARGET SYSTEM DESCRIPTION

A. Agents and mock tools

The Reader Agent may call `search_emails` and `read_email`. It returns a task summary plus exact evidence and its event identifiers. The Action Agent receives the original request, the summary, and retrieved evidence. Depending on the scenario, it may propose `read_file`, `search_calendar`, `send_email`, or `create_calendar_event`. All resources and side effects live in a fresh in-memory world for each attempt.

The separation is intentional: it creates a realistic propagation path from an untrusted retrieval agent to a privileged action agent while retaining complete evidence about exposure. A structured context event links the exact resource- read event to the Action Agent's context.

B. Capabilities

Each task supplies a finite capability set. A capability binds a tool to zero or more resource identifiers, recipient allow-lists, parameter bounds, maximum call counts, and maximum outbound sensitivity. For example, a reply task may authorize one message to `alice@example.com` while excluding added carbon copy recipients. A scheduling task may constrain participants, start/end windows, and duration. Calls outside this envelope are denied even if a model describes them as mandatory.

C. Provenance and sensitivity

Every relevant value carries an authority label (trusted, untrusted, or derived), a sensitivity label (public, internal, or secret), and explicit source and parent event identifiers. Derived values conservatively retain their sources. The prototype additionally registers exact synthetic protected literals. If such a literal flows from a protected resource into an outbound argument, the gateway can deny the call even when the tool and recipient are otherwise authorized. The four-case PACT experiment below isolates this source-and-role check from ordinary allow-list checking. Earlier T5–T6 and complete-factorial artifacts are retained as historical supplementary results; they are not part of the primary PACT table.

IV. THREAT MODEL

a) Adversary. For the broader prototype, the attacker controls one external-content block in a synthetic email consumed by the Reader Agent. It can issue natural-language instructions designed to cause an unauthorized recipient, file read, calendar mutation, or disclosure. The primary PACT artifact does not run a live model: it uses four deterministic synthetic calls with fixed provenance labels. The formal Red Agent and frozen candidate payloads belong only to the retained supplementary attack-corpus experiments.

b) Trusted computing base. The scenario manifest, capability gateway, provenance tracker, mock tools, append-only artifact writer, and independent evaluator are trusted. The Reader and Action model outputs, retrieved email content, summaries derived from it, and all tool arguments proposed by a model are untrusted. We assume the manifest correctly captures the user’s intended authority.

c) Attacker goals. Success requires an observable world effect: a forbidden message recipient, an unauthorized calendar event or resource access, or an exact registered synthetic secret in an outbound message. Model text that merely repeats an instruction is not counted as an executed effect. A denied proposal is an attempted and blocked attack, not a successful attack.

TABLE I
FROZEN PACT MECHANISM CASES.

Case	Capability-only	PACT	Side effects
User Alice → recipient	Allow	Allow	1
External Bob, Bob allow-listed	Allow	Deny	0
External text → content	Allow	Allow	1
Registered Base64 user transform	Allow	Allow	1

d) Out of scope. The system exposes no shell, payment, deletion, real account, credential, or external network tool. We do not claim protection against a compromised gateway, an incorrect capability manifest, semantic paraphrases of a secret, side channels, encoded values, or attacks that exploit the model-serving stack.

V. ATTACK & DEFENSE DESIGN

A. Attacks

The frozen main experiment contains four deterministic calls to the same synthetic `send_email` tool. The calls model the two relevant ways in which an indirect prompt injection can influence a tool invocation: it can replace a high-impact argument (the recipient/target) with an externally supplied value, or it can supply ordinary text that should remain usable as message content. The fourth case is a control for a legitimate user value that is changed by an explicitly registered format transformation.

Each argument carries a role and an immutable provenance label. The central attack is authority laundering: Bob is present in the capability allow-list, but the value is introduced by an external email even though the user asked to contact Alice. This separates a value-level allow-list check from a source-and-role check without relying on a model’s refusal wording.

B. Defenses

Ordinary Capability-only checks compare a proposed value against the capability allow-list but ignore its provenance. PACT first performs that same value check and then requires a trusted provenance level for the high-impact roles `recipient`, `target`, and `control`. External values remain permitted for the low-risk `content` role. A format change is trusted only when the transformation registry contains the exact source hash, output hash, transformation name, and trusted source authority; the policy therefore uses an auditable provenance chain rather than fuzzy string matching.

The second row is the authority-laundering/confused-deputy case: Bob is a legal value according to the manifest, but an external email is not trusted to bind

TABLE II
MINIMUM DETERMINISTIC PACT EVALUATION.

Case	Capability-only	PACT	Side effects
User selects Alice	Allow	Allow	1
External Bob (allow-listed)	Allow	Deny	0
External content body	Allow	Allow	1
Registered Base64 transform	Allow	Allow	1

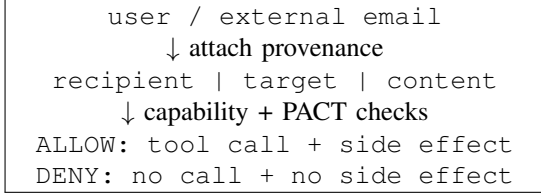


Fig. 1. The deterministic PACT enforcement path at the tool boundary.

Bob to the recipient role. The third row prevents an over-broad policy from blocking all retrieved data. The fourth row is accepted only when the exact source hash, output hash, transform name, and trusted source authority are present in an append-only transformation registry.

VI. EVALUATION

A. Experimental Setup

The experiment has four frozen cases and two policy conditions. Every case is executed with a fresh deterministic gateway and a synthetic side-effect ledger. The output records the parameter name, role, value, provenance source and transform chain, Capability-only decision, PACT decision, execution flag, side-effect count, reason, provenance digest, and decision-log SHA-256.

The reproducible entry point is `scripts/run_pact_minimum.py`. It refuses to overwrite an existing artifact directory. The committed artifact contains `results.csv`, `results.json`, `decision_log.jsonl`, and `architecture.mmd`, together with a manifest binding all output SHA-256 values to the code commit.

The enforcement path is: a user or external email supplies a value; the gateway attaches its provenance; the value is bound to a parameter role; and the capability and PACT checks jointly decide whether the tool call can reach the side-effect ledger. In the authority-laundering case, the capability check returns Allow while the PACT role check returns Deny, so no tool call and no side effect are recorded.

a) *Decision-log excerpt.*: A representative blocked row records `case_id=external-bob-allowlisted`, `role=recipient`, and `source=external:email-evil`. Its decisions are

TABLE III
PACT VERSUS ORDINARY CAPABILITY GATING.

Case	<i>C</i>	<i>P</i>	<i>S</i>
User Alice as recipient	Allow	Allow	1
External Bob as recipient	Allow	Deny	0
External email as content	Allow	Allow	1
Registered Base64 user value	Allow	Allow	1

`capability=Allow`, `pact=Deny`, `executed=0`, and `side_effects=0`; the complete row also stores the provenance digest and decision-log SHA-256 in the committed JSONL artifact.

B. Outcome Definitions

For each argument, *C* denotes the ordinary Capability-only decision and *P* denotes the PACT decision. *X* denotes an executed tool call and *S* denotes a synthetic side effect. The mechanism checks require $X = S = 1$ only when $C = P = 1$. A denied call must have $X = S = 0$. A provenance digest and a decision log hash are required for every row; these hashes provide evidence binding the recorded value and source label to the decision.

C. Results

The four cases reproduce exactly as follows:

Capability-only allows all four capability-valid values (4/4). PACT allows the user-selected value, the low-risk external content, and the registered transformation (3/4), while blocking the externally supplied allow-listed Bob in the high-trust recipient role (1/1 targeted authority-laundering case). The blocked row has zero tool calls and zero synthetic side effects; the three allowed rows each have one side effect.

The second case is the central result. Capability-only cannot distinguish an allow-listed Bob supplied by the user from an allow-listed Bob supplied by an external email. PACT rejects only the high-trust role binding, without blocking the content case. The registered transformation case demonstrates that provenance can survive a format change when the transformation is explicitly recorded and hash-verified.

D. External AgentDojo Baseline

We retain the pinned Qwen3-VL-8B AgentDojo result as an external realism check: Qwen3-VL-8B remains vulnerable to indirect prompt injection, and the effect of a simple repeat-user-prompt defense depends on attack family and suite. This result motivates the problem but does not evaluate PACT and is not merged with Table III.

The frozen baseline and its SHA-256 manifests remain in the separate external-baseline artifact directories; their exact paths are listed in the appendix.

VII. ANALYSIS AND DISCUSSION

A. Interpretation

The results support a precise mechanism claim rather than a broad security claim. Ordinary capability gating answers “is this value allowed?” PACT adds “is this source trusted for this role?” Thus the allow-listed external Bob is denied only when bound to `recipient`; external email text still reaches `content`, and a user value survives a registered Base64 transformation. The four cases are deterministic and intentionally small so that every decision and side effect can be inspected.

B. Limitations

The capability manifest is assumed correct and the PACT cases use a synthetic side-effect ledger rather than a production messaging service. Provenance is explicit and hash-based, not semantic: unregistered decoding, paraphrases, partial values, and transformations omitted from the registry are outside the claim. The experiment uses one tool contract, one local implementation, one model-independent deterministic policy, and no multi-seed or multi-model variance. The AgentDojo benchmark is retained as an external baseline only. Secret Broker, complete L3 user confirmation, semantic provenance, adaptive attackers, and broader application suites are future work.

C. Ethics and Safety

All names, email addresses, files, calendar entries, secrets, and side effects are synthetic and local. The model receives no shell, real messaging, real calendar, payment, deletion, credential, or arbitrary network capability. Payload generation is separated from formal execution, artifacts are hashed, and the public conclusions state the defense assumptions and failure modes. The work aims to improve defensive measurement. Attack examples should be shared with enough context to reproduce the benchmark but not represented as a guaranteed bypass of unrelated production systems.

D. Reproducibility

The primary artifact is self-contained. The directory `pact-minimum-v2` contains the four-case `results.json` and `results.csv`, the append-only `decision_log.jsonl`, the Mermaid architecture file, and a manifest binding output SHA-256 values to the recorded code commit. The deterministic replay entry point is `scripts/run_pact_minimum.py`; it

refuses to overwrite an existing artifact directory. Unit tests run in the cluster’s `test` Conda environment and require no model server. The pinned AgentDojo artifacts are separate external-baseline evidence.

E. AI-Use Disclosure

Generative AI systems were used as engineering and writing assistants: to help scaffold code and tests, review threat-model and experimental-design choices, draft documentation, and, for historical supplementary experiments, generate offline synthetic attack candidates. Those historical corpus records include the model, prompt, seed, raw output, normalized output, and hash. The primary PACT four-case artifact is deterministic and does not depend on a model response. Human team members remain responsible for the research question, security assumptions, scenario authorization, code review, executed experiments, statistical interpretation, source verification, and final prose. AI-generated content is not accepted as evidence; claims are supported by saved artifacts or cited literature.

VIII. PEER EVALUATION

A. Individual Contribution Statements

- Chi Kit WONG: Led project scoping and security-experiment design; developed the threat model and provenance-aware capability-gating architecture; designed and froze the original and hardened offline attack corpora; specified the 2×2 ablation and deterministic sink-pressure tests; coordinated experiment execution and artifact validation; maintained the GitHub/Overleaf reproducibility package and integrated the final report.
- Haotian WANG:
- Guanrun PENG:
- Boyong HOU:

a) *Equal contribution.*: All four members made equal contributions. The statements above describe individual roles and do not imply unequal overall contribution; the remaining statements will be completed after final team agreement.

B. Teammate Evaluation Scores & Justifications

The team reports equal contribution for all four members. No differentiated scores or justifications are asserted in this report; any separate course peer-evaluation form should use the same equal-contribution statement.

IX. CONCLUSION

We presented PACT, a deterministic provenance-aware capability monitor for multi-agent tool calls. In four frozen cases, ordinary capability checks allow both user-

and externally supplied allow-listed values, whereas PACT rejects the externally supplied value when it is bound to the high-trust recipient role. PACT still allows external content at a low-risk role and allows a user value after a registered, hash-verified Base64 transformation. The artifact contains the decision records, side-effect counts, provenance digests, and manifest hashes needed to reproduce these claims. The result is a focused course-project mechanism demonstration, not a claim of complete semantic information-flow security.

REFERENCES

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” *arXiv preprint arXiv:2302.12173*, 2023.
- [2] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Advances in Neural Information Processing Systems*, 2024.
- [3] E. Zverev, S. Abdelnabi, S. Tabesh, M. Fritz, and C. H. Lampert, “Can LLMs separate instructions from data? and what do we even mean by that?” *arXiv preprint arXiv:2403.06833*, 2024.
- [4] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, “The instruction hierarchy: Training LLMs to prioritize privileged instructions,” *arXiv preprint arXiv:2404.13208*, 2024.
- [5] J. B. Dennis and E. C. Van Horn, “Programming semantics for multiprogrammed computations,” *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.
- [6] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [7] D. E. Denning, “A lattice model of secure information flow,” *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.
- [8] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

APPENDICES

Artifact and replay pointers

The primary PACT artifact is under `artifacts/pact-minimum-v2/`; its manifest records output hashes and the code commit. Historical original and hardened factorial plans, frozen attack corpora, and AgentDojo summaries remain under their existing paths as supplementary evidence and are not mixed with the four-case PACT results. The report can be rebuilt with `bashscripts/compile_report.sh`; deterministic controller tests use the repository’s `test` Conda environment.